



Java IO Stream基础

北京理工大学计算机学院
金旭亮

概述



为了简化开发，Java将文件粗略地分为两大类：文本文件和“文本文件”之外的其他文件，为两者分别提供不同的文件存取组件，而不是试图设计一套复杂的组件，同时支持两种类型的文件存取。

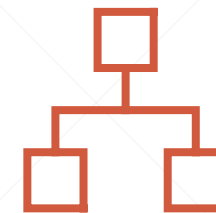


JDK为文本文件提供了一套“字符”相关的组件，以“**字符 (char)**”为基本数据存取单元。其他类型的文件，被看成是二进制数据的序列，则主要通过各种“**流 (Stream)**”组件来进行存取，其基本单元是“**字节 (byte)**”。

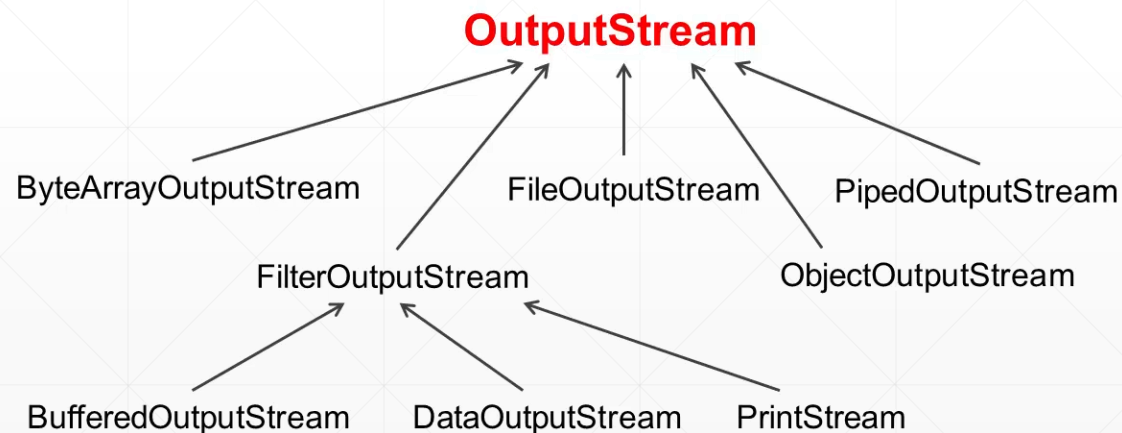
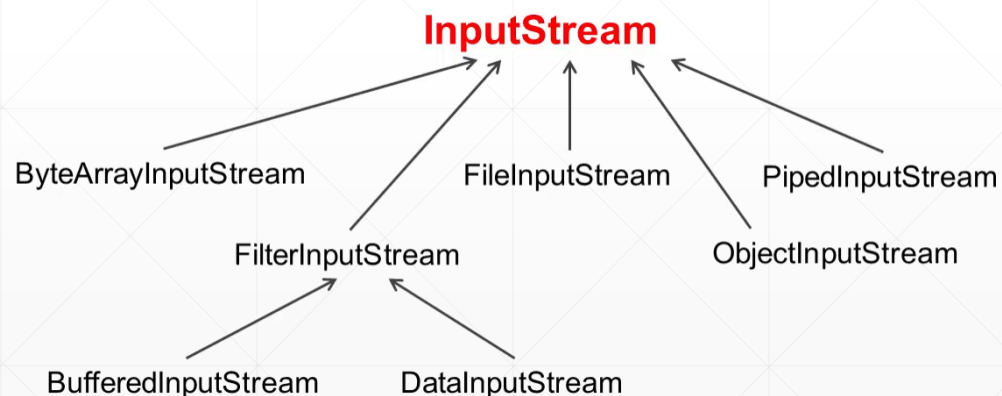


本讲主要介绍如何使用“流”来进行存取文件。

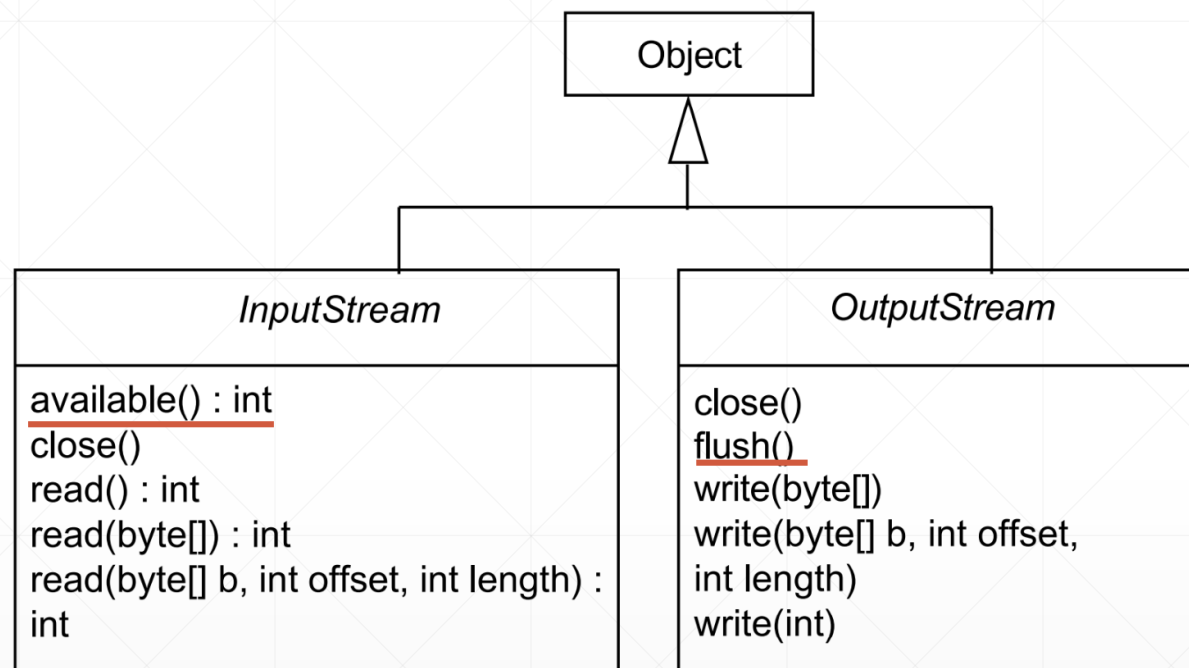
字节流



字节流，以字节（byte）为单位读取或写入原始数据流。最顶层的基类是 `InputStream` 和 `OutputStream`，这两个都是抽象类，在实际开发中，使用的其实是它的子类。



InputStream和OutputStream的主要成员



读与写，都是以字节为基本单位的。



写文件时，使用`flush()`保证数据写入文件，读文件时，可使用`available()`作为循环判断条件，不断地读取文件。

使用InputStream中读取数据

```
abstract int read() throws IOException
```

一次读一个整数

```
int read(byte[] b, int offset,int length)  
throws IOException
```

一次读length个字节，放到b这个字节数组中，从offset这个位置开始存放。

本方法底层是通过循环调用read()方法实现的

当检测到流的结束时，这两个方法都返回-1

使用OutputStream写入数据

```
abstract void write(int) throws IOException
```

将一个整数，写入到流中

```
void write(byte[] b, int offset, int length)  
    throws IOException
```

从字节数组b的第offset个位置开始，向流中写入length个字节。在底层，其实是通过循环调用write()方法实现的。

基于流的编程模型 (以文件流为例)

```
FileInputStream fis = null;
try
{
    fis = new FileInputStream("image.jpg");
    // Read bytes from file.
    int _byte;
    while ((_byte = fis.read()) != -1) // -1 signifies EOF
        ; // Process _byte in some way.
}
catch (IOException ioe)
{
    // Handle exception.
}
finally
{
    if (fis != null)
    {
        try
        {
            fis.close();
        }
    }
}
```

1

打开文件，创建流对象

2

通过流存取文件

3

用完后，关闭流

这些都是使用流存取文件的固定套路了！

注意:



在java.io中，针对流的读和写操作，都是阻塞（blocking）的。



非阻塞（blocking）读写，需要使用java.nio包中的组件。

简化使用流的代码-1

由于Java引入了受控异常的这种特性，导致Java关闭流的代码非常麻烦，为了简化，Java 7引入了try-with-resources特性，能自动关闭相应的流对象：

将流对象封装到try后面的表达式中.....

```
try (FileInputStream in = new FileInputStream(filename)) {  
    // Read data  
} catch (FileNotFoundException e) {  
    ...  
} catch (IOException e) {  
    ...  
}
```

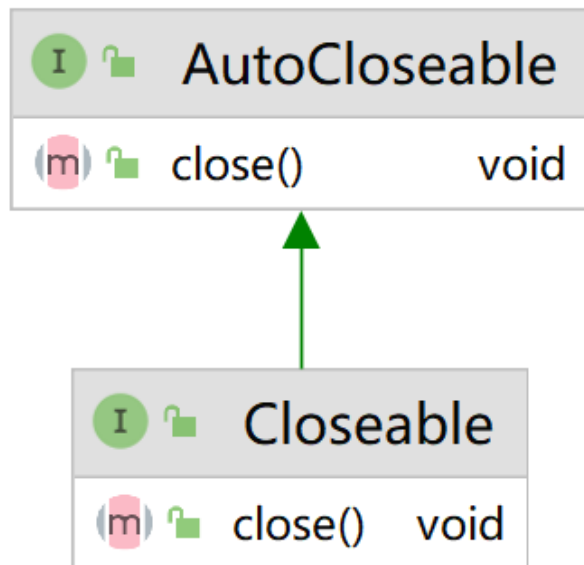
当执行流程离开try语句块时，自动地关闭它所监控的流对象.....

简化使用流的代码-2

可以将多个流对象都放到`try`后面的表达式中，使用分号隔开多条语句就行：

```
try (FileInputStream in = new FileInputStream(filename);  
    FileOutputStream out = new FileOutputStream(filename)) {  
    // Read data  
} catch (FileNotFoundException e) {  
    ...  
} catch (IOException e) {  
    ...  
}
```

AutoCloseable和Closeable接口



如果一个类实现`Closeable`接口，则表明它可以用于`try with resources`结构中。

JDK 7+, `InputStream/OutputStream`实现了`AutoCloseable`接口，所以可以在`try...catch`中使用而不需要显式关闭。

写完之后，要flush()...

由于操作系统会缓存文件数据，因此，程序中调用OutputStream的write系列方法向文件写入数据时，应该随后立即调用flush()方法，通知操作系统将数据立即写入底层文件，不然，有可能会

出现数据未真正存入文件的情形出现。



调用OutputStream的close()方法关闭流时，会自动地flush()而无需手工调用。

自定义“可以释放”的资源类示例

```
public class MyAutoClosable implements Closeable {  
    @Override  
    public void close() throws IOException {  
        System.out.println("Close方法被调用");  
    }  
  
    public static void main(String[] args) {  
        try (var obj = new MyAutoClosable()) {  
            System.out.println(obj.getClass().getName() + "实例化。");  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Run: MyAutoClosable x

↑ "C:\Program Files\Java\jdk-17\bin\java.exe"
↓ MyAutoClosable实例化。
↺ Close方法被调用
↻ Process finished with exit code 0